SMS SPAM AND SCAM MESSAGE DETECTION

Software Requirements Specification • Minimum Viable Product • System Workflow

Project type AIML Mini-Project	**Proposed stack** Python • Scikit-learn • NLTK • Streamlit

Prepared as a scoped implementation document for an academic mini-project.

1. Project Overview

Project title: SMS Spam and Scam Message Detection Using Natural Language Processing and Machine Learning

The proposed system analyzes the text of an SMS message and classifies it as legitimate, spam, or potentially fraudulent. It uses Natural Language Processing (NLP) to prepare message text, TF-IDF to convert text into numerical features, and supervised machine learning models to predict a category. A lightweight Streamlit interface will allow users to enter messages and view predictions.

Problem statement. SMS spam and scam messages can expose users to unwanted promotions, phishing links, impersonation, and fraudulent requests. Simple keyword filters may miss changing message patterns or incorrectly flag legitimate messages. This project explores whether machine learning can identify suspicious messages from their textual characteristics and provide a useful classification result.

2. Software Requirements Specification (SRS)

2.1 Functional requirements

ID	Requirement	Description
FR01	Message input	Allow users to enter or paste an SMS message.
FR02	Text preprocessing	Normalize and clean message text before analysis.
FR03	Feature extraction	Convert text into numerical features using TF-IDF.
FR04	Classification	Predict the message category using a trained model.
FR05	Confidence display	Show a model probability or confidence estimate where supported and appropriately calibrated.
FR06	Risk indicators	Highlight potentially suspicious URLs, urgent language, or requests for sensitive information.
FR07	Result display	Show the predicted category and a brief explanation.
FR08	Model evaluation	Report accuracy, precision, recall, F1-score, and a confusion matrix.

2.2 Non-functional requirements

ID	Requirement	Description
NFR01	Performance	A prediction should normally complete within a few seconds on a standard laptop.
NFR02	Usability	The interface should be simple and understandable.
NFR03	Reliability	Handle empty, very long, or invalid input safely.
NFR04	Maintainability	Separate preprocessing, training, prediction, and interface code.
NFR05	Privacy	Do not permanently store user-entered messages by default.
NFR06	Compatibility	Support Windows, Linux, and macOS with compatible Python dependencies.
NFR07	Evaluation	Assess performance using class-sensitive metrics, not accuracy alone.

2.3 Hardware requirements

Component	Minimum suggested requirement
Processor	Intel Core i3 or equivalent

Component	Minimum suggested requirement
RAM	4 GB
Storage	Approximately 2 GB available
Operating system	Windows, Linux, or macOS
Internet	Useful for downloading datasets and dependencies; not required for local prediction after setup

2.4 Software requirements

Software / library	Purpose
Python 3.10 or later	Primary programming language
Pandas and NumPy	Dataset handling and numerical operations
Scikit-learn	TF-IDF, model training, and evaluation
NLTK (optional)	Tokenization and selected text preprocessing
Streamlit	Lightweight web interface
Matplotlib / Seaborn	Charts and confusion matrix visualization
Joblib	Saving and loading the trained model and vectorizer
Jupyter Notebook or VS Code	Development environment

3. Dataset Requirements

A suitable starting point is the **SMS Spam Collection Dataset** from the UCI Machine Learning Repository: UCI SMS Spam Collection. It contains messages labeled as ham (legitimate) or spam.

Important scope note: the UCI dataset is primarily a binary ham/spam dataset. It does not, by itself, provide reliable labels for a separate scam class. To train a genuine three-class model, the project must obtain or create an additional scam dataset with verified labels and consistent labeling rules. Do not simply relabel all spam as scam.

Recommended initial approach: build and evaluate a reliable binary classifier first. Add a third class only if enough suitable, labeled scam examples can be collected. If the project retains binary ML classification, suspicious scam indicators can be displayed separately as rule-based warnings, clearly distinguished from the model prediction.

4. Minimum Viable Product (MVP)

4.1 MVP objective

Build a small working application that accepts a user-provided SMS message, applies the same preprocessing and feature extraction used during training, predicts a category, and displays the result. The MVP should demonstrate the end-to-end machine learning workflow without requiring mobile integration, cloud hosting, or access to a user's SMS inbox.

4.2 Core MVP features

Feature	Description	Priority
SMS input	Paste or type a message into the interface.	Must have
Preprocessing	Normalize text consistently with training data.	Must have
TF-IDF extraction	Transform the message using the saved vectorizer.	Must have
ML prediction	Classify the message using the selected trained model.	Must have
Result display	Show the predicted class clearly.	Must have
Model evaluation	Show metrics and confusion matrix for test data.	Must have
Risk indicators	Flag suspicious links or phrases as supporting signals.	Should have
Confidence estimate	Display a probability only if the model supports it; explain that it is not a guarantee.	Could have
Session history	Optionally retain recent results only during the current session.	Could have

4.3 Proposed models

Algorithm	Role
Multinomial Naive Bayes	Lightweight baseline that is commonly used for text classification.
Logistic Regression	Strong linear baseline for sparse TF-IDF text features.
Random Forest	Optional comparison with a tree-based model; may be less efficient on high-dimensional sparse text.

Recommended starting point: compare Multinomial Naive Bayes and Logistic Regression. Select the model using test-set performance, with particular attention to spam/scam recall and false positives. Do not select a model using accuracy alone.

4.4 Interface screens

Screen	Content
Home	Project title, short explanation, message input area, and Analyze button.
Prediction result	Predicted category, optional confidence estimate, and suspicious indicators.
Model evaluation	Accuracy, precision, recall, F1-score, and confusion matrix for the held-out test set.

4.5 Out of scope

The MVP will not automatically read, block, or delete SMS messages; connect to telecom providers; send mobile notifications; deploy a cloud service; or provide guaranteed fraud detection. Multilingual support and advanced deep learning are future enhancements.

5. System Workflow

5.1 Training workflow

Step	Activity	Output
1	Collect a labeled SMS dataset and inspect class distribution.	Raw labeled dataset
2	Remove duplicates, handle missing values, and inspect labels.	Clean dataset
3	Normalize text using a consistent preprocessing pipeline.	Processed messages
4	Split into training and testing subsets using stratification.	Train/test sets
5	Fit TF-IDF on training text only; transform test text using the fitted vectorizer.	Numerical feature matrices
6	Train Naive Bayes and Logistic Regression models.	Trained candidate models
7	Evaluate on the held-out test set.	Metrics and confusion matrix
8	Select a model and save it with the fitted vectorizer.	Model artifacts

5.2 Prediction workflow

Step	Activity	Output
1	User enters or pastes an SMS message.	Raw input
2	Validate that the input is not empty and apply the same text normalization.	Validated message
3	Transform the message using the saved TF-IDF vectorizer.	Feature vector
4	Pass the feature vector to the saved classifier.	Predicted class
5	Optionally inspect URL patterns or suspicious phrases using transparent rules.	Supporting indicators
6	Display the category and supporting information.	Result screen

5.3 Architecture

User interface (Streamlit) → Input validation and preprocessing → Saved TF-IDF vectorizer → Trained classifier → Prediction and optional risk indicators → Result visualization

The training pipeline is separate from the prediction interface. The fitted vectorizer and classifier must be saved and loaded together so that new messages are represented in exactly the same feature space as the training data.

6. Suggested Project Structure

SMS_Spam_Detection/
■■■ dataset/
■ ■■■ sms_dataset.csv
■■■ models/
■ ■■■ spam_classifier.pkl
■ ■■■ tfidf_vectorizer.pkl
■■■ src/
■ ■■■ preprocessing.py
■ ■■■ feature_extraction.py
■ ■■■ train_model.py

■ ■■■ prediction.py
■■■ app.py
■■■ requirements.txt
■■■ README.md

7. Evaluation and Expected Outcomes

The project is expected to produce a functional local application that classifies SMS messages using a trained machine learning model. Evaluation should include:

• Accuracy: overall proportion of correct predictions.
• Precision: proportion of predicted positive messages that are actually positive.
• Recall: proportion of actual positive messages detected.
• F1-score: balance between precision and recall.
• Confusion matrix: breakdown of correct and incorrect predictions by class.

Use a stratified held-out test set. Avoid data leakage by fitting text preprocessing steps and TF-IDF only on the training data. If messages from the same source or near-duplicates occur, consider grouping or deduplicating them before splitting. Report class-wise metrics, especially because spam datasets are often imbalanced.

8. Development Plan

Phase	Activity	Deliverable
1	Review the problem and collect the dataset.	Problem definition and dataset
2	Clean and explore the data.	Preprocessed dataset and class analysis
3	Build the TF-IDF pipeline.	Feature extraction pipeline
4	Train and compare candidate models.	Trained models and comparison
5	Evaluate the selected model.	Metrics and confusion matrix
6	Build the Streamlit interface.	Interactive local application
7	Integrate, test, and document.	Working MVP and final report

9. Limitations and Future Enhancements

Limitations: predictions depend on the quality and representativeness of labeled data; text-only models may miss context, sender identity, and link behavior; new scam patterns may not be recognized; and a model probability is not a guarantee that a message is safe or fraudulent.

Future enhancements: add a properly labeled scam class, support multiple languages, analyze shortened or suspicious URLs, compare contextual language models, add explainability, and explore optional mobile integration.

10. Final Deliverable

A Python-based Streamlit application that accepts an SMS message, processes it using a saved NLP pipeline, predicts its category using a trained machine learning model, and presents the result alongside model evaluation evidence. The initial implementation should prioritize a well-evaluated binary spam classifier; three-class scam classification should be included only when suitable labeled data is available.

Dataset reference: UCI Machine Learning Repository, SMS Spam Collection.
https://archive.ics.uci.edu/dataset/228/sms+spam+collection